Admin 前端测试指南（PropOS）

> 适用版本：Vue 3 + Vite + Pinia + Element Plus（`admin/` 目录）
> 文档日期：2026-04-28

一、测试分层架构

```
┌──────────────────────────────────────┐
│        E2E 集成测试（Playwright）      │
│  关键用户流程：登录、资产浏览、合同、财务 │
├──────────────────────────────────────┤
│      组件测试（@testing-library/vue）   │
│   LoginView / ContractsView / 表单交互 │
├──────────────────────────────────────┤
│    Store 单元测试（@pinia/testing）    │
│  useAuthStore / useUsersStore / 资产存储 │
├──────────────────────────────────────┤
│      API 层单元测试（axios-mock-adapter）│
│  client.ts 拦截器 / modules/ 函数     │
└──────────────────────────────────────┘
```

二、测试技术栈

工具	用途
`vitest`	测试运行器（与 Vite 同生态，共享配置）
`jsdom`	浏览器 DOM 环境模拟
`@testing-library/vue`	Vue 3 组件挂载 + 用户行为查询
`@testing-library/user-event`	真实用户交互模拟（输入、点击）
`@pinia/testing`	`createTestingPinia`（隔离 store 副作用）
`@vitest/coverage-v8`	V8 代码覆盖率报告
`axios-mock-adapter`	拦截 axios 实例，测试 client.ts 拦截器链

npm 脚本：

```
pnpm test              # 运行所有单元测试（watch 模式）
pnpm test:run          # CI 模式，单次运行
pnpm test:coverage     # 生成覆盖率报告
```

三、目录结构

```
admin/
├── vitest.config.ts                    # Vitest 独立配置
└── src/
    ├── test-utils/
    │   └── setup.ts                    # 全局测试初始化（Element Plus 注册）
    ├── api/
    │   └── client.test.ts              # HTTP 客户端拦截器测试
    ├── stores/
    │   ├── auth.test.ts                # 认证 store 测试
    │   ├── assets.test.ts              # 资产 store 测试
    │   └── users.test.ts               # 用户 store 测试
    ├── views/
    │   └── auth/
    │       └── LoginView.test.ts       # 登录页组件测试
    └── router/
        └── index.test.ts               # 路由守卫测试
```

四、API 层测试规范

4.1 client.ts 核心测试场景

JWT 注入：

```
// 请求拦截器必须读取 localStorage 中的 access_token
// 并以 Bearer xxx 格式写入 Authorization 头
it('注入 JWT 到 Authorization 头', async () => {
  localStorage.setItem('access_token', 'token-abc')
  mock.onGet('/api/test').reply(200, { data: {} })
  await apiGet('/api/test')
  // 断言请求头包含 Bearer token-abc
})
```

401 Token 刷新链路：

```
// 第一次 401 → 调用 /api/auth/refresh → 用新 token 重试原请求
it('401 触发 token 刷新后重试请求', async () => {
  // 1. 首次请求返回 401
  // 2. refresh 端点返回新 token
  // 3. 原请求以新 token 重试，返回 200
})
```

并发 401 防重复刷新：

```
// 多个并发请求同时 401，只发送一次 refresh，所有请求共享新 token
it('并发 401 只刷新一次 token', async () => {
  // 3 个并发请求同时 401 → refresh 只被调用 1 次 → 3 个请求都成功
})
```

刷新失败 → 强制登出：

```
// refresh 端点本身失败 → 清空 token + 跳转 /login + 抛出 ApiError
it('刷新失败时跳转登录页', async () => {
  // refresh 返回 401 → router.replace('/login') 被调用
  // → 抛出 ApiError('UNAUTHORIZED')
})
```

认证端点不触发刷新循环：

```
// /api/auth/login 和 /api/auth/refresh 本身 401 直接报错，不触发刷新
it('登录端点 401 不循环刷新', async () => {})
```

错误信封解析：

```
// 后端 { error: { code: 'XXX', message: '...' } } 转换为 ApiError
it('将后端错误信封转换为 ApiError', async () => {
  mock.onGet('/api/test').reply(404, { error: { code: 'NOT_FOUND', message: '资源不存在' } })
  await expect(apiGet('/api/test')).rejects.toMatchObject({
    code: 'NOT_FOUND',
    message: '资源不存在',
    statusCode: 404,
  })
})
```

4.2 API modules/ 测试规范

- 每个 module 函数测试其 **URL 路径** 和 **请求参数格式**
- Mock 方案：`vi.mock('@/api/client')` 后用 `vi.mocked(apiGet).mockResolvedValue(...)`
- 不测试 HTTP 实现细节，只测试函数签名和参数组装

五、Store 层测试规范

5.1 测试骨架模板

```
import { setActivePinia, createPinia } from 'pinia'
import { beforeEach, describe, it, expect, vi } from 'vitest'

// Mock API 模块（在 store 层不测 HTTP 实现）
vi.mock('@/api/modules/xxx')

describe('useXxxStore', () => {
```

```
  beforeEach(() => {
    setActivePinia(createPinia())
    localStorage.clear()
  })

  it('初始状态', () => {
    const store = useXxxStore()
    expect(store.list).toEqual([])
    expect(store.loading).toBe(false)
    expect(store.error).toBeNull()
  })

  it('fetchList 成功：填充 list 和 meta', async () => { /* ... */ })
  it('fetchList 失败：设置 error.value', async () => { /* ... */ })
})
```

5.2 useAuthStore 测试矩阵

方法	场景	断言
`login()`	成功	tokens 存入 localStorage，profile 非空，跳转 /dashboard
`login()`	ApiError	`error.value = e.message`，不 logout
`login()`	403	`clearTokens()` 被调用
`login()`	loading 变化	`false → true → false`
`fetchMe()`	成功	`profile.value` 被填充
`fetchMe()`	非 403 错误	自动调用 `logout()`
`fetchMe()`	403	不调用 logout，重新抛出
`logout()`	revokeSession=true	调用 `apiLogout`，清空 profile 和 token
`logout()`	revokeSession=false	不调用 `apiLogout`
`isLoggedIn`	computed	profile 非空时为 true
`role`	computed	返回 profile.role

5.3 useAssetOverviewStore 测试矩阵

方法	场景	断言
`fetchAll()`	成功	`list` 填充楼栋，`overview` 填充统计
`fetchAll()`	聚合计算	`buildingOccupancy` 按 building_id 正确统计出租率
`fetchAll()`	non_leasable 排除	非可租状态不计入 total
`fetchAll()`	失败	`error.value` 为 ApiError.message
`exportUnits()`	成功	触发浏览器下载（URL.createObjectURL）
`exportUnits()`	失败	`error.value` 被设置，异常重新抛出

5.4 useUsersStore 测试矩阵

方法	场景	断言
`load()`	成功	`list` 和 `meta` 填充
`load()`	带参数	filters 被合并，API 以新参数调用
`load()`	失败	`error.value` 被设置
`create()`	成功	调用 `load()` 刷新列表，返回 detail
`create()`	失败	`error.value` 设置，异常重新抛出
`update()`	成功	调用 `load()` 刷新列表
`update()`	失败	`error.value` 被设置

六、组件层测试规范

6.1 测试骨架模板

```
import { render, screen } from '@testing-library/vue'
import userEvent from '@testing-library/user-event'
import { createTestingPinia } from '@pinia/testing'
import { vi } from 'vitest'

const renderComponent = (storeOverrides = {}) =>
  render(LoginView, {
    global: {
      plugins: [createTestingPinia({ createSpy: vi.fn, initialState: storeOverrides })],
    },
  })
```

6.2 LoginView 测试场景

场景	操作	断言
渲染	无操作	显示邮箱输入、密码输入、登录按钮
空提交	点击登录	显示"请输入邮箱"错误
邮箱格式错误	输入非法邮箱后 blur	显示"请输入有效的邮箱地址"
loading 状态	`authStore.loading = true`	按钮显示 loading 且不可点击
错误展示	`authStore.error = '账号不存在'`	el-alert 显示对应文字
成功登录	填写有效表单并提交	`authStore.login` 以正确参数被调用
忘记密码跳转	点击"忘记密码"	router.push('/forgot-password') 被调用

七、路由守卫测试规范

7.1 测试场景

场景	操作	断言
未登录访问受保护路由	navigate to `/dashboard`	重定向 `/login?redirect=/dashboard`
未登录访问公开路由	navigate to `/login`	不重定向，访问成功
已登录访问任意路由	token 存在，navigate to `/assets`	访问成功
forgot-password 公开	navigate to `/forgot-password`	不需要 token

7.2 路由测试要点

- 使用 `createMemoryHistory`（不依赖浏览器 URL）
- `localStorage.setItem('access_token', ...)` 模拟登录态
- 测试 `router.currentRoute.value.name` 验证跳转结果

八、测试覆盖率目标

层次	目标覆盖率	关注重点
`api/client.ts`	≥ 90%	所有拦截器分支、错误路径
`stores/`	≥ 85%	所有 action 的成功和失败路径
`views/auth/`	≥ 80%	表单交互、错误展示
`router/`	≥ 90%	守卫所有条件分支

九、运行命令参考

```
# 单次运行所有单元测试
pnpm test:run

# 生成 HTML 覆盖率报告（输出到 coverage/）
pnpm test:coverage

# 仅运行指定文件
pnpm test:run src/stores/auth.test.ts

# Watch 模式（开发时使用）
pnpm test
```

十、Mock 策略汇总

测试对象	Mock 方案
`client.ts` 拦截器	`axios-mock-adapter` 包裹 `http` 实例
Store 内 API 调用	`vi.mock('@/api/modules/xxx')`
组件内 Store	`createTestingPinia({ createSpy: vi.fn })`
`localStorage`	`beforeEach(() => localStorage.clear())`
`vue-router`	`vi.mock('vue-router')` 或 `createMemoryHistory`
`URL.createObjectURL`	`vi.stubGlobal('URL', ...)`